

Database Systems (DV3,DAT5,SW5) Transaction & Concurrency Control

Yumeng Song and Juan Manuel Rodriguez

Department of computer science, Aalborg University

Fall 2025



AALBORG
UNIVERSITY

DBMS Internals

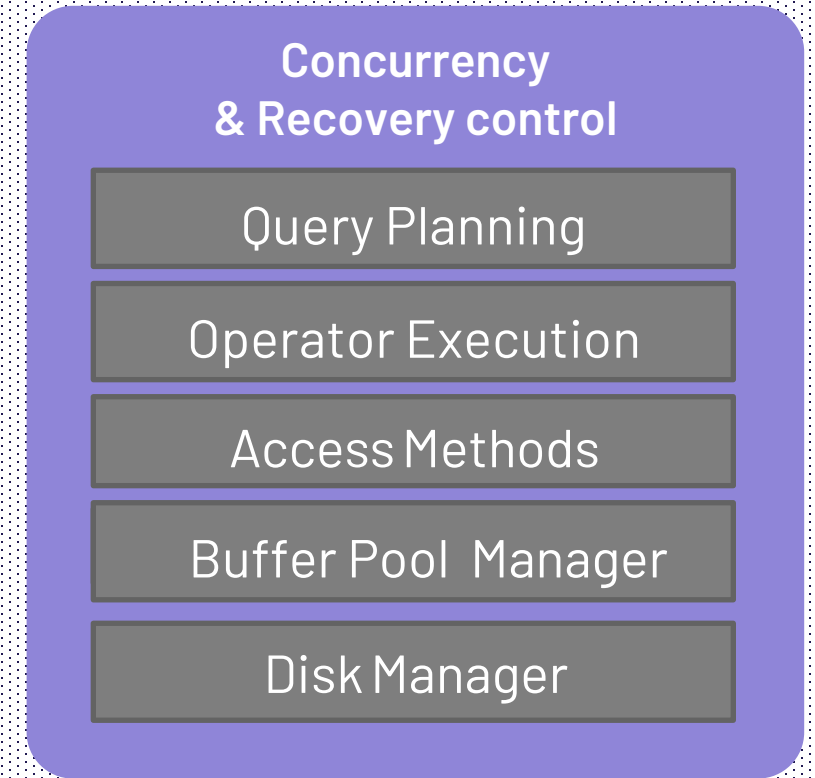
- ▶ We know how to use a DBMS.
- ▶ We know how to store and access data efficiently in a DBMS.
- ▶ We know **how to execute queries efficiently**.
- ▶ How do we **make sure we run the query** in the **most efficient way** among the alternatives available? What **about multiple concurrent queries?**





Keeping the Data safe in the DBMS

- How do we design a DB and organize it in a DBMS so that the queries are executed efficiently?
- What makes a query slow or fast? Expensive to compute or efficient?
- What ensures that our **data is stored safely and correctly** also when **multiple queries** are running at the same time?



Outline

1. Concurrency Control

- ▶ Transactions ★
- ▶ ACID Properties ★
- ▶ Serial Execution ★
- ▶ Interleaving Transactions

2. Scheduler

- ▶ Serial Schedule
- ▶ Equivalent Schedule
- ▶ Serializable Schedule ★
- ▶ Conflicting Transactions
- ▶ Conflict Serializability ★

3. Scheduling & Isolation Level

- ▶ Locks: Exclusive/Shared ★
- ▶ 2-Phase Locking basics ★
- ▶ 2-Phase Locking details
- ▶ Strong Strict 2-Phase locking ★
- ▶ Deadlocks ★
- ▶ Isolation Levels

4. Durability

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

Chapters 17, 18 without 18.10, 19 without 19.7-19.10

- from Database System Concepts 7th edition ; Silberschatz et al.

Intro to Database Systems (15-445/645), Andy Pavlo CMU

- <https://15445.courses.cs.cmu.edu/fall2022/>



Motivation

Two users are buying plane tickets
and they try to book the same seat on the plane.

How do we ensure that only one gets the seat?
(Avoid race condition)

**Concurrency
Control**
*(Consistency
& Isolation)*

A user is sending the money to buy a new laptop
but there is a power failure in the server room and the server
shuts-down.

How do we establish the correct state when power gets back?

**Recovery
Control**
*(Atomicity
& Durability)*



Concurrency Control and Recovery

- The user workflow can require multiple queries
- We talk about a “Transaction” (txns)

A transaction:

It is the basic unit of change in a DBMS

A transaction is the execution of a sequence of one or more operations (e.g., SQL queries) on a database to perform some higher-level function.

→ Partial transactions are not allowed!

Transactions need to ensure a set of four properties (*see later*)

A : Atomicity

C : Consistency

I : Isolation

D : Durability

Example of use of Transaction

Book the seat on a plane

User

1. Check available Seats
2. Check price of seat 7B
3. Check price of seat 18B
4. Book seat 7B

System

- > 7B, 13B, and 18B are available
- > 70 DKK
- > 120 DKK
- > **[Start Transaction]**
1. Check 7B is available
 2. Subtract 70 DKK from **\$user** Account
 3. Mark 7B Booked by **\$user**
- [End Transaction]**

Each one of these can also be a transaction, but...

This transaction can fail here

We don't want someone booking 7B while we are here



Concurrency Problem

- The user workflow can require multiple queries
- We talk about a "Transaction"
- The system receives multiple transactions in parallel
- Some transactions may fail and then we need to reset the status of the DB
- Some transactions may want to read and write the same values

Unpractical Solution:

Queue all Transactions, Execute one by one,
Create a copy of the data to recover if a
transaction fails



**We want to exploit
parallelism as much as
possible**



Property for a Transaction

- All queries within a transaction should be executed or the transaction is Aborted
 - During execution of a transaction the DB can be in invalid state, but never at the end of it!
 - For simplicity: We represent a transaction as a series of Read and Write operations on some "data object"
 - T1: R(A); R(B); W(C);
 - T2: R(A); W(B); R(C); W(A);
- These two transactions want to both Read/Write on the same data objects (can be same tuple, same attribute, etc.)

In SQL a transaction starts with "BEGIN" and ends with "COMMIT" or "ABORT"

- ABORT reset the status to before any operation *OF THIS TXN* was executed



Properties for a Transaction: ACID

Atomicity

All actions in txn happen, or none happen.

"All or nothing..."

if a debit is made successfully from one account, the corresponding credit is made to the other account.

Consistency

If each txn is consistent and the DB starts consistent, then it ends up consistent.

"It looks correct to me..."

the total value of funds across accounts is the same at the start and end of each transaction.

Isolation

Execution of one txn is isolated from that of other txns.

"All by myself..."

another transaction sees the transferred funds in one account or the other, but not in both, nor in neither.

Durability

If a txn commits, its effects persist.

"I will survive..."

the changes made to each account will not be reversed.



Serial Execution

Assume $A=1000$ and $B=1000$

T1: $A=A-100$; $B=B+100$

T1: $R(A)$; $W(A)$; $R(B)$; $W(B)$;

T2: $A=A*1.06$; $B=B*1.06$

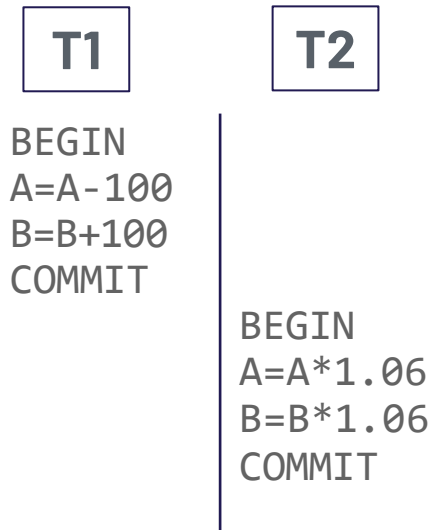
T2: $R(A)$; $W(A)$; $R(B)$; $W(B)$;

**In both schedules
the 2 transactions run
one after the other, in isolation,
and the result is consistent**

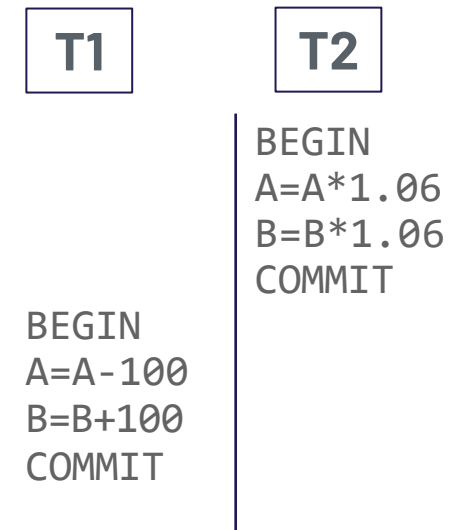
TIME

2 Serial Schedules

Schedule A T1 then T2



Schedule B T2 then T1



A=954, B=1166

A=960, B=1160

$$A+B=2000*1.06= 2120$$



Interleaving Transactions

The user workflow can require multiple queries;

Multiple users want to run queries at the same time;

- We interleave txns to maximize concurrency.**

- Slow disk/network I/O.

- Multi-core CPUs.

**When one txn stalls because of a resource (e.g., I/O),
another txn can continue executing and make forward progress.**



Interleaving Execution: Good Schedule

Assume $A=1000$ and $B=1000$

T1: $A=A-100$; $B=B+100$

T1: $R(A); W(A); R(B); W(B);$

T2: $A=A*1.06$; $B=B*1.06$

T2: $R(A); W(A); R(B); W(B);$

In the first schedule the 2 transactions are interleaved, but the result is equivalent as if they were running in isolation! This is "serializable"

2 Equivalent Schedules

Schedule A Interleave T1 and T2

BEGIN
 $A=A-100$

$B=B+100$
COMMIT

BEGIN
 $A=A*1.06$

$B=B*1.06$
COMMIT

Schedule B T1 then T2

BEGIN
 $A=A-100$
 $B=B+100$
COMMIT

BEGIN
 $A=A*1.06$
 $B=B*1.06$
COMMIT

$A=954, B=1166$

$A=954, B=1166$

$$A+B=2000*1.06= 2120$$



Interleaving Execution: Bad Schedule

Assume $A=1000$ and $B=1000$

T1: $A=A-100$; $B=B+100$

T1: $R(A)$; $W(A)$; $R(B)$; $W(B)$;

T2: $A=A*1.06$; $B=B*1.06$

T2: $R(A)$; $W(A)$; $R(B)$; $W(B)$;

In the first schedule the 2 transactions are interleaved, but the result is not equivalent as if they were running in isolation!
This is "not serializable"

Schedules NOT Equivalent

Schedule A
Interleave T1 and T2

BEGIN
 $A=A-100$

BEGIN
 $A=A*1.06$
 $B=B*1.06$
COMMIT

$B=B+100$
COMMIT

$A=954, B=1160$

Schedule B
T1 then T2

BEGIN
 $A=A-100$
 $B=B+100$
COMMIT

BEGIN
 $A=A*1.06$
 $B=B*1.06$
COMMIT

$A=954, B=1166$

Inconsistent Result!

$A+B=2000*1.06= 2120$
but $954+1160 = 2114$

Outline

1. Concurrency Control

- ▶ Transactions ★
- ▶ ACID Properties ★
- ▶ Serial Execution ★
- ▶ Interleaving Transactions

2. Scheduler

- ▶ Serial Schedule
- ▶ Equivalent Schedule
- ▶ Serializable Schedule ★
- ▶ Conflicting Transactions
- ▶ Conflict Serializability ★

3. Scheduling & Isolation Level

- ▶ Locks: Exclusive/Shared ★
- ▶ 2-Phase Locking basics ★
- ▶ 2-Phase Locking details
- ▶ Strong Strict 2-Phase locking ★
- ▶ Deadlocks ★
- ▶ Isolation Levels

4. Durability

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

Chapters 17, 18 without 18.10, 19 without 19.7-19.10

- from Database System Concepts 7th edition ; Silberschatz et al.

Intro to Database Systems (15-445/645), Andy Pavlo CMU

- <https://15445.courses.cs.cmu.edu/fall2022/>

Outline

1. Concurrency Control

- ▶ Transactions ★
- ▶ ACID Properties ★
- ▶ Serial Execution ★
- ▶ Interleaving Transactions

2. Scheduler

- ▶ Serial Schedule
- ▶ Equivalent Schedule
- ▶ Serializable Schedule ★
- ▶ Conflicting Transactions
- ▶ Conflict Serializability ★

3. Scheduling & Isolation Level

- ▶ Locks: Exclusive/Shared ★
- ▶ 2-Phase Locking basics ★
- ▶ 2-Phase Locking details
- ▶ Strong Strict 2-Phase locking ★
- ▶ Deadlocks ★
- ▶ Isolation Levels

4. Durability

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

Chapters 17, 18 without 18.10, 19 without 19.7-19.10

- from Database System Concepts 7th edition ; Silberschatz et al.

Intro to Database Systems (15-445/645), Andy Pavlo CMU

- <https://15445.courses.cs.cmu.edu/fall2022/>



Interleaving Execution: DBMS modelling

Assume $A=1000$ and $B=1000$

T1: $A=A-100$; $B=B+100$

T1: $R(A)$; $W(A)$; $R(B)$; $W(B)$;

T2: $A=A*1.06$; $B=B*1.06$

T2: $R(A)$; $W(A)$; $R(B)$; $W(B)$;

The DBMS when handling transactions, cannot see the real operations!

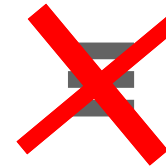


Schedule A Interleave T1 and T2

BEGIN
 $R(A)$
 $W(A)$

BEGIN
 $R(A)$
 $W(A)$
 $R(B)$
 $W(B)$
COMMIT

$R(B)$
 $W(B)$
COMMIT



Schedule B T1 then T2

BEGIN
 $R(A)$
 $W(A)$
 $R(B)$
 $W(B)$
COMMIT

BEGIN
 $R(A)$
 $W(A)$
 $R(B)$
 $W(B)$
COMMIT



Serial and Serializable Schedules

Serial Schedule: A schedule that does not interleave operations of different transactions

Equivalent Schedule: Two schedule that when executed starting from the same state end up leaving the database in the same state

Serializable Schedule: A schedule that is equivalent to a serial schedule
(equivalent to a serial execution of transactions)

This means we do not care about timestamps of transactions!



Interleaving Execution: Conflicts

Assume $A=1000$ and $B=1000$

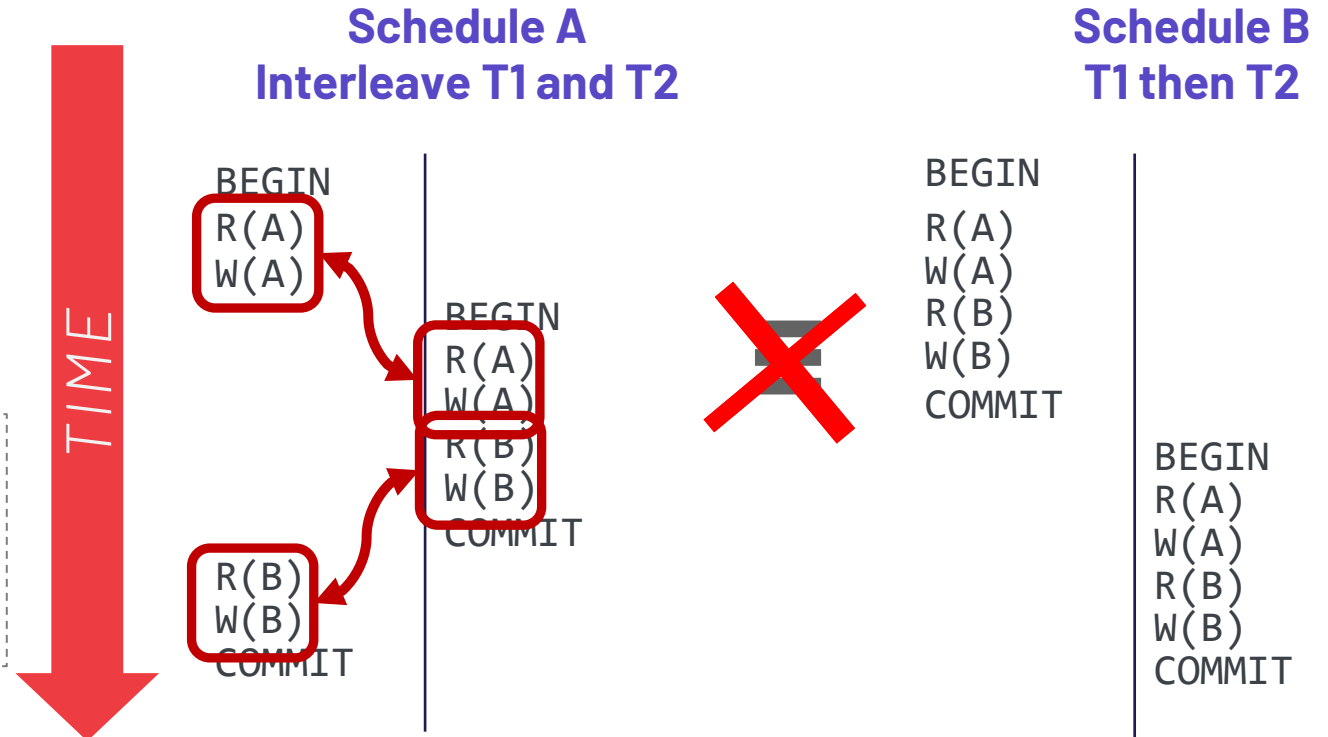
T1: $A=A-100$; $B=B+100$

T1: $R(A)$; $W(A)$; $R(B)$; $W(B)$;

T2: $A=A*1.06$; $B=B*1.06$

T2: $R(A)$; $W(A)$; $R(B)$; $W(B)$;

These Two Transactions want to execute operations that are in conflict





Determine Serializable Schedules

Goal: Given a set of transactions, find out whether a given schedule is serializable

Conflict: two operations are in conflict if:

1. they are executed by different transactions
2. they access the **same object** and **at least one of them is a write**

Whenever there is a conflict there is the
potential of a data integrity issue

Type of Conflicts:

- Read-Write (R-W)
- Write-Read (W-R)
- Write-Write (W-W)

Given a conflict we identify **execution anomalies**



No Conflict

ISOLATION CRITERIA:

If a transaction reads a value and does not edit the value, all subsequent reads should provide the same value.

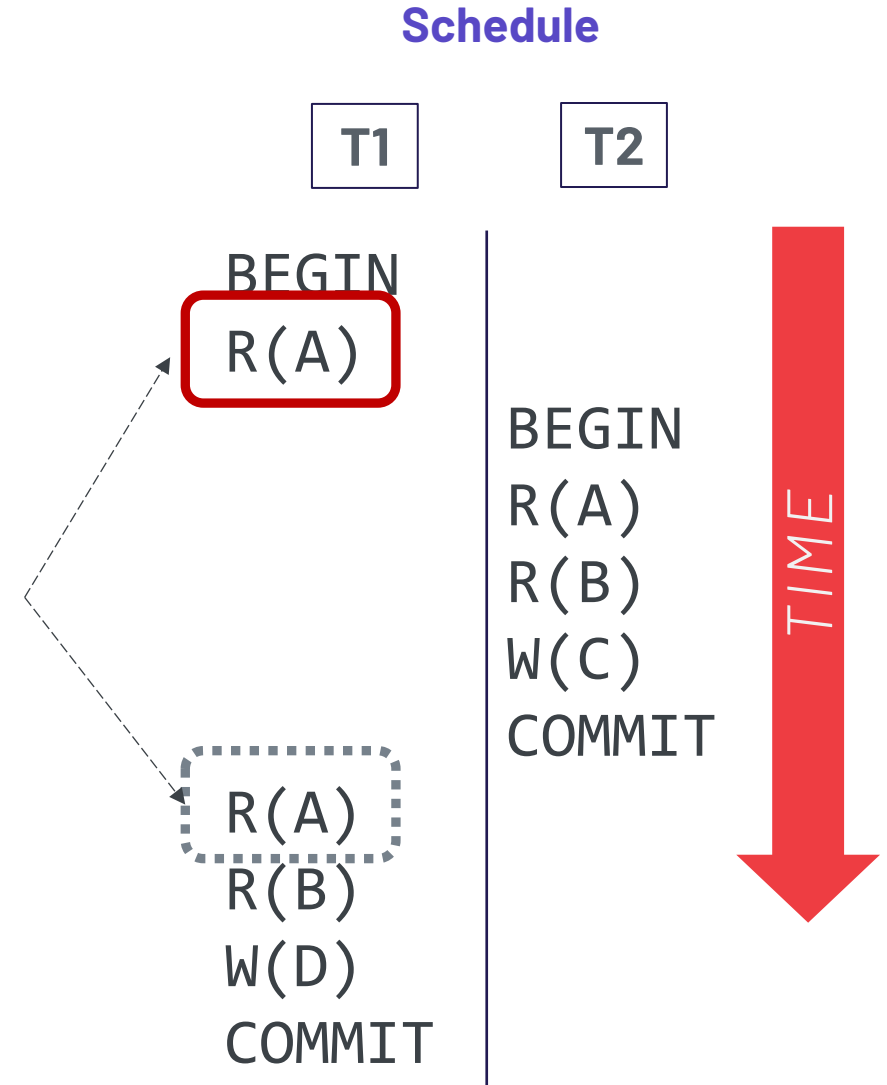
These two transactions **ONLY READ** on shared resources and **WRITE** on **DISTINCT RESOURCES**

NO CONFLICT!

We do not care in which order they are executing their operations, the result is always fine.

T2 does not affect T1:

These 2 reads will get the same value





Conflicts R-W → Anomaly: Unrepeatable Read

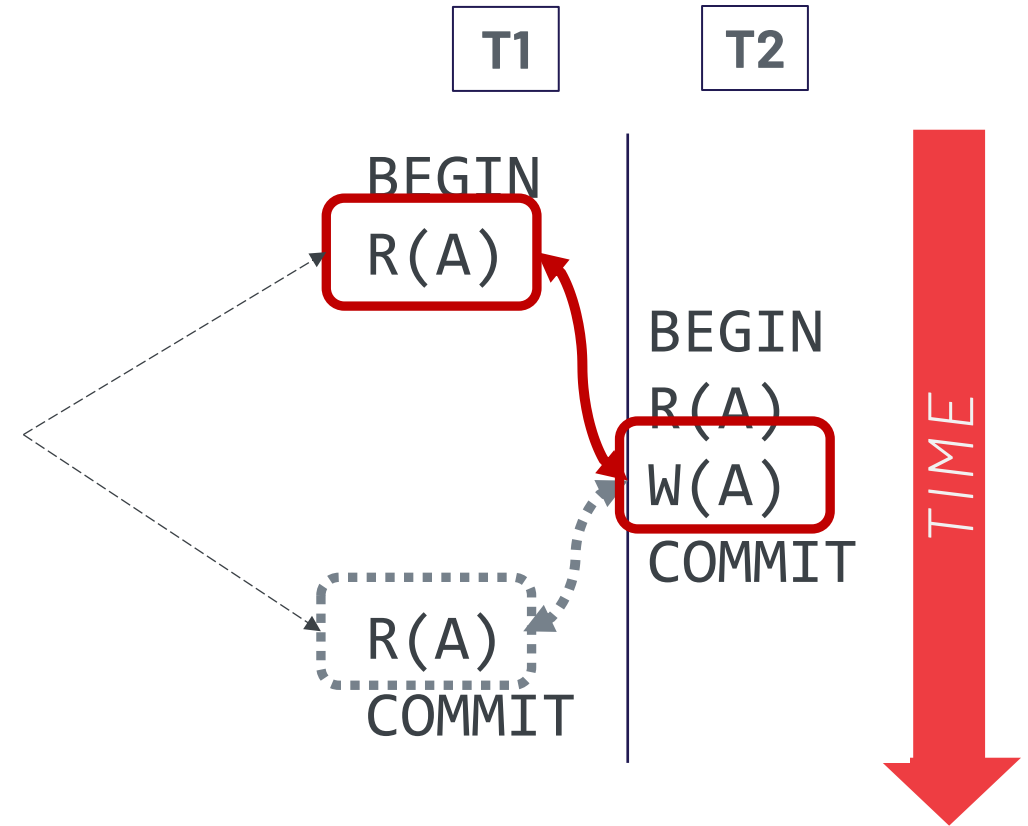
Schedule

ISOLATION CRITERIA:

If a transaction reads a value and does not edit the value, all subsequent reads should provide the same value

*There is a conflict if in two distinct transactions **one reads and one writes** on the same resource: if the first one reads again, it causes **unrepeatable reads in the first***

T2 affects T1:
Breaks isolation
These 2 reads will get different values



Unrepeatable Read:

One Txn gets different values when reading the same object multiple times

Example:

T1: How much money do I have?

T2: Take money from A



Conflicts W-R → Anomaly: Dirty Read

ISOLATION CRITERIA:

If a transaction reads a value, the value should be consistent and durable

*There is a conflict if in two distinct transactions **one writes and one reads** on the same resource: if the first one aborts, the written value is not valid, it causes **dirty reads in the second***

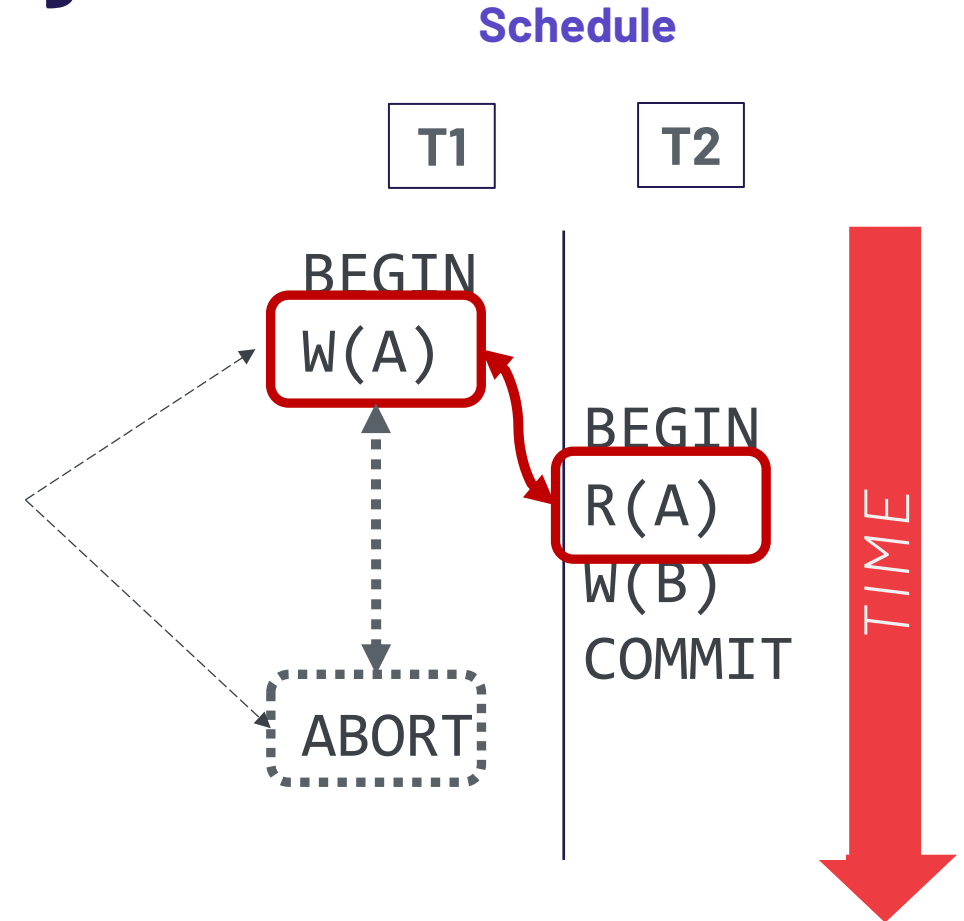
Dirty Read:

Txn reads a value that has not been committed and could be rolled back

Note: In some system we use ROLLBACK instead of ABORT

T1 Fails:

The Write is rolled back, but T2 is not aware of that!



Example:

T1: Move money from A to C

T2: Move money from A to B



Conflicts W-W → Anomaly: Lost Update

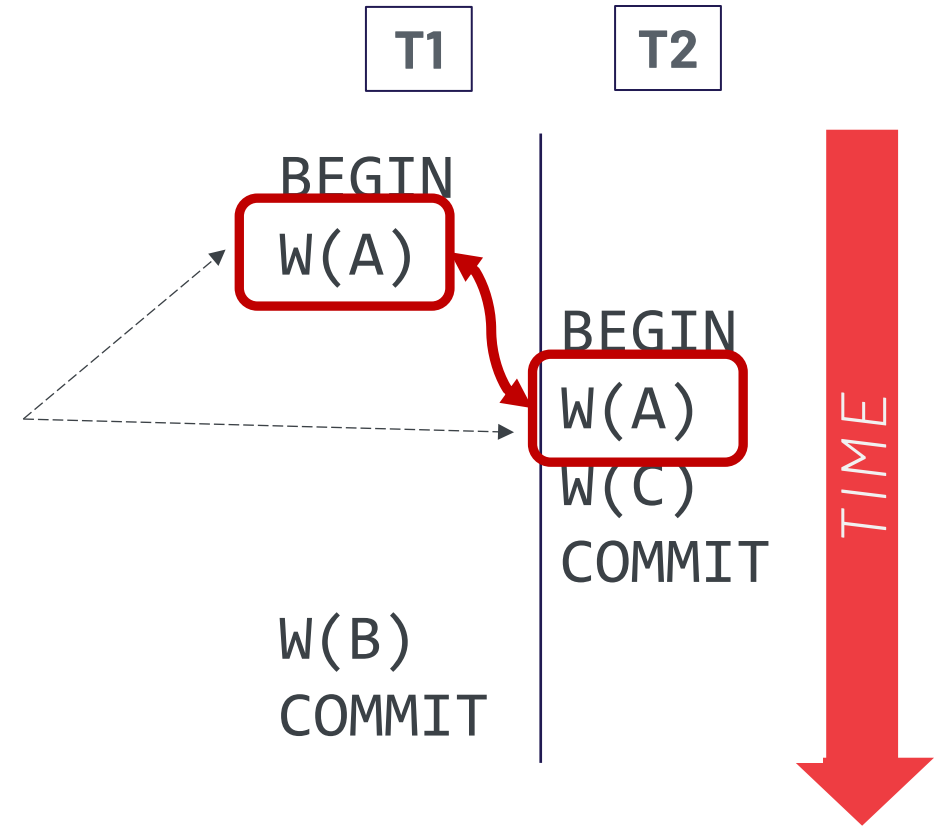
Schedule

ISOLATION CRITERIA:

If a transaction commits a value, the value should be consistent and durable before another transaction STARTS

*There is a conflict if in two distinct transactions **one writes and the other also writes** on the same resource: the second write operation overwrites the first, so the first update is a **lost update***

This is a race condition only one of them should do this



Lost Update:

One Txn overwrites data from another uncommitted Txn

Example:

T1: Set the grade of student A to 12

T2: Set the grade of student A to 02



Determine Serializable Schedules in Practice

Goal: Given a set of transactions, given that we identify conflicts, find out whether a given schedule is serializable.

There will always be conflicts, but some of them are “correct” because equivalent to serial operations!

Different Level of “Serializability” :

1. Conflict serializability ← **Most DBMS Try to do this**
2. View Serializability ← No way to implement it in practice

Whenever there is a conflict there is the *potential* of a data integrity issue, in a serial schedule even if there are conflicts, we know there are no anomalies!

Type of Conflicts:

- Read-Write (R-W)
- Write-Read (W-R)
- Write-Write (W-W)



Conflict Serializable Schedules

Two schedules are conflict equivalent if and only if both apply:

- A. They involve the same actions of the same transactions
- B. Every pair of conflict actions is ordered in the same way

A schedule $\underline{S}$ is **Conflict Serializable** if it is **conflict equivalent** to some serial schedule (no matter which one)

- Therefore: it is possible to transform $\underline{S}$ into a serial schedule by swapping consecutive non-conflicting operations of different transactions

Remember: serial schedules will still have conflicts, but they are accepted conflicts



Serial/Serializable and Conflicts

Given 2 transactions

T1: W(A); R(C); R(B); W(B);

T2: R(C); R(B); W(B); W(C);

Conflicts depend on the order of operations!

Conflicts Schedule A

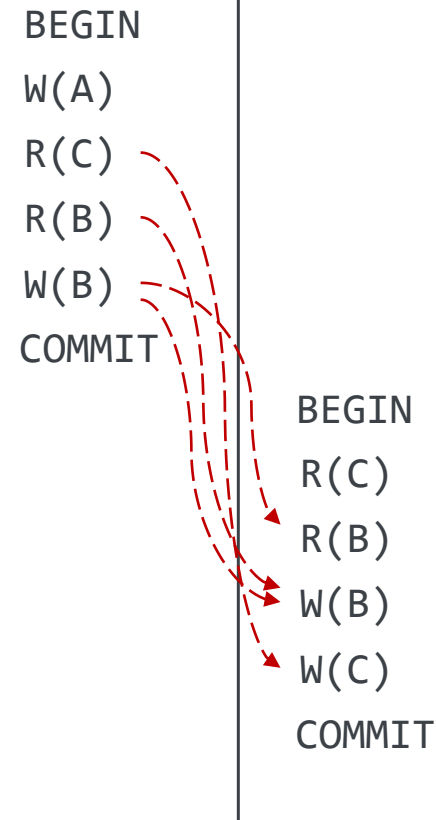
T1.RC $\rightsquigarrow$ T2.WC

T1.RB $\rightsquigarrow$ T2.WB

T1.WB $\rightsquigarrow$ T2.RB

T1.WB $\rightsquigarrow$ T2.WB

Schedule A First T1 then T2





Serial/Serializable and Conflicts

Given 2 transactions

T1: W(A); R(C); R(B); W(B);

T2: R(C); R(B); W(B); W(C);

Conflicts depend on the order of operations!

Conflicts Schedule A

T1.RC $\rightsquigarrow$ T2.WC

T1.RB $\rightsquigarrow$ T2.WB

T1.WB $\rightsquigarrow$ T2.RB

T1.WB $\rightsquigarrow$ T2.WB

Conflicts Schedule B

T2.WC $\rightsquigarrow$ T1.RC

T2.RB $\rightsquigarrow$ T1.WB

T2.WB $\rightsquigarrow$ T1.RB

T2.WB $\rightsquigarrow$ T1.WB



Schedule A First T1 then T2

BEGIN

W(A)

R(C)

R(B)

W(B)

COMMIT

BEGIN

R(C)

R(B)

W(B)

W(C)

COMMIT

Schedule B First T2 then T1

BEGIN

R(C)

R(B)

W(B)

W(C)

COMMIT

BEGIN

W(A)

R(C)

R(B)

W(B)

COMMIT

Both are serial schedule, so both will leave the DB in a consistent state! But they are not conflict equivalent



Serial/Serializable and Conflicts

Given 2 transactions

T1: W(A); R(C); R(B); W(B);

T2: R(C); R(B); W(B); W(C);

Conflicts depend on the order of operations!

Conflicts Schedule A

T1.RC $\rightsquigarrow$ T2.WC

T1.RB $\rightsquigarrow$ T2.WB

T1.WB $\rightsquigarrow$ T2.RB

T1.WB $\rightsquigarrow$ T2.WB

Conflicts Schedule B

T2.WC $\rightsquigarrow$ T1.RC

T2.RB $\rightsquigarrow$ T1.WB

T2.WB $\rightsquigarrow$ T1.RB

T2.WB $\rightsquigarrow$ T1.WB

Conflicts Schedule C

T1.RC $\rightsquigarrow$ T2.WC

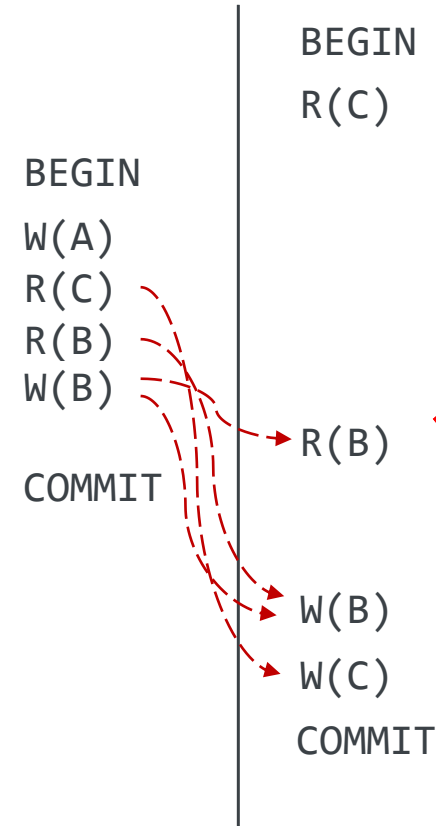
T1.RB $\rightsquigarrow$ T2.WB

T1.WB $\rightsquigarrow$ T2.RB

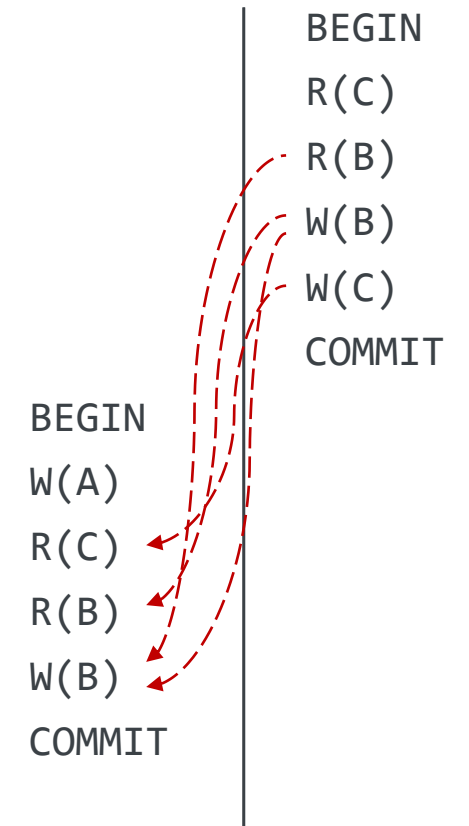
T1.WB $\rightsquigarrow$ T2.WB



Schedule C Interleave T2 & T1



Schedule B First T2 then T1



The two schedule C&B are NOT conflict equivalent



Serial/Serializable and Conflicts

Given 2 transactions

T1: W(A); R(C); R(B); W(B);

T2: R(C); R(B); W(B); W(C);

Conflicts depend on the order of operations!

Conflicts Schedule A

T1.RC $\rightsquigarrow$ T2.WC

T1.RB $\rightsquigarrow$ T2.WB

T1.WB $\rightsquigarrow$ T2.RB

T1.WB $\rightsquigarrow$ T2.WB

Conflicts Schedule B

T2.WC $\rightsquigarrow$ T1.RC

T2.RB $\rightsquigarrow$ T1.WB

T2.WB $\rightsquigarrow$ T1.RB

T2.WB $\rightsquigarrow$ T1.WB

Conflicts Schedule C

T1.RC $\rightsquigarrow$ T2.WC

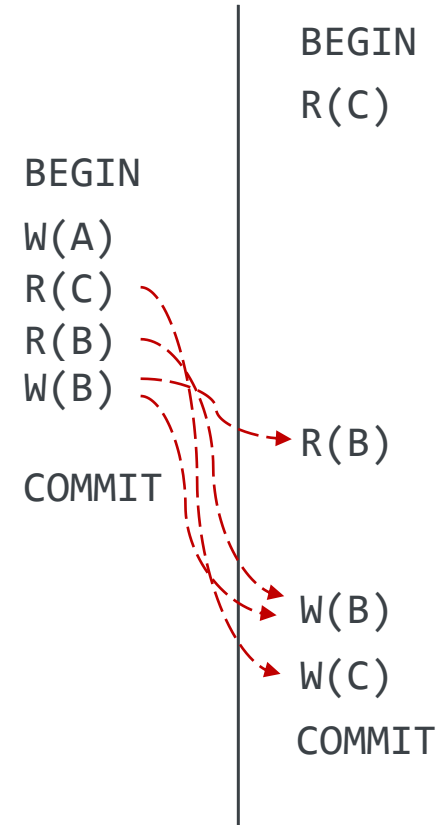
T1.RB $\rightsquigarrow$ T2.WB

T1.WB $\rightsquigarrow$ T2.RB

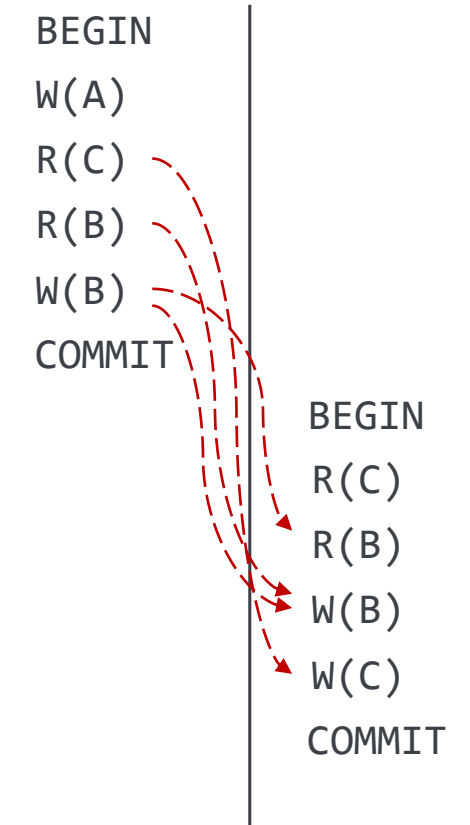
T1.WB $\rightsquigarrow$ T2.WB



Schedule C Interleave T2 & T1



Schedule A First T1 then T2



The two schedule C&A are conflict equivalent $\rightarrow$ C is conflict serializable



Is it Conflict Serializable?

Schedule D Interleave T1 & T2

BEGIN
W(A)

R(C)
R(B)

W(B)
COMMIT

BEGIN
R(C)

R(B)
W(B)

W(C)
COMMIT

≡ ?

Schedule A First T1 then T2

BEGIN
W(A)
R(C)
R(B)
W(B)
COMMIT

BEGIN
R(C)
R(B)
W(B)
W(C)
COMMIT

Schedule B First T2 then T1

BEGIN
R(C)
R(B)
W(B)
W(C)
COMMIT

BEGIN
W(A)
R(C)
R(B)
W(B)
COMMIT

Conflicts Schedule D

T1.RC $\rightsquigarrow$ T2.WC
T1.RB $\rightsquigarrow$ T2.WB
T2.RB $\rightsquigarrow$ T1.WB
T2.WB $\rightsquigarrow$ T1.WB

Conflicts Schedule A

T1.RC $\rightsquigarrow$ T2.WC
T1.RB $\rightsquigarrow$ T2.WB
T1.WB $\rightsquigarrow$ T2.RB
T1.WB $\rightsquigarrow$ T2.WB

Schedule D is not equivalent to A neither to B!

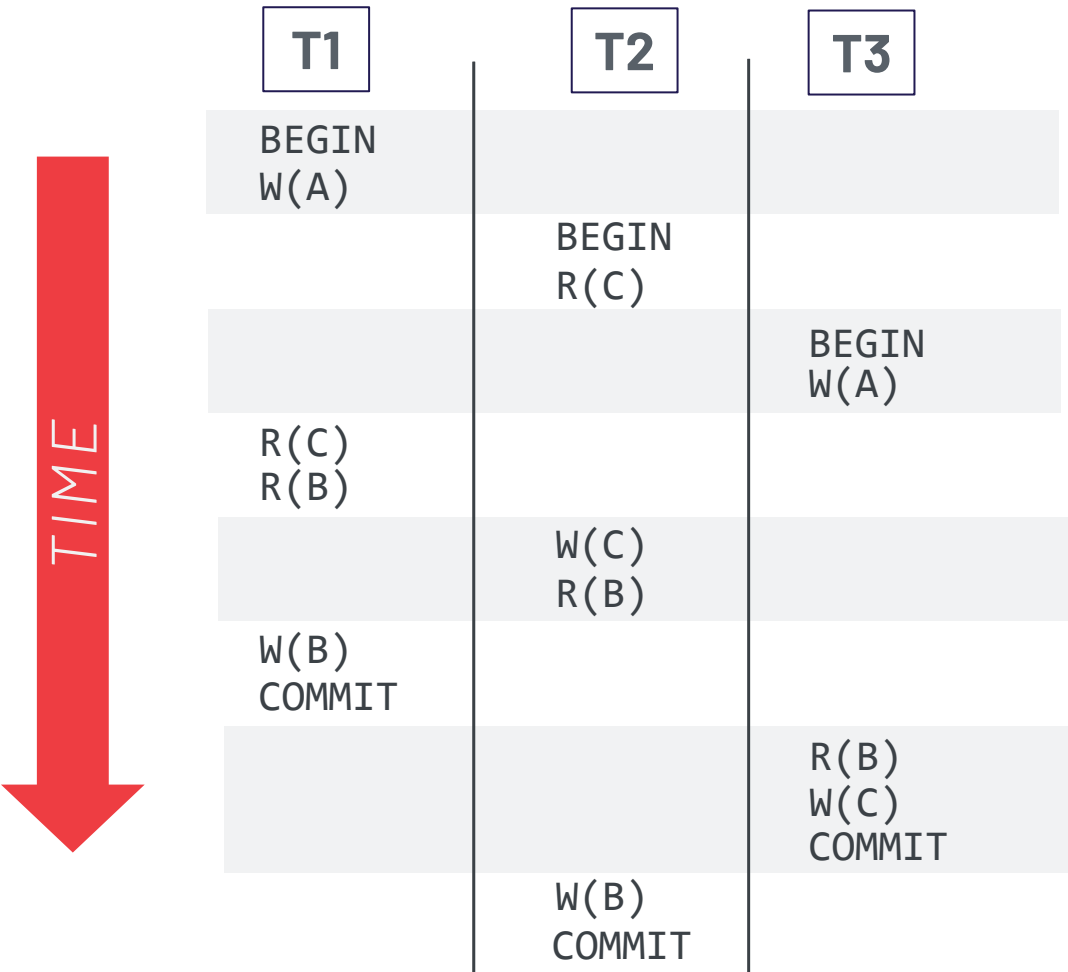
Conflicts Schedule B

T2.WC $\rightsquigarrow$ T1.RC
T2.RB $\rightsquigarrow$ T1.WB
T2.WB $\rightsquigarrow$ T1.RB
T2.WB $\rightsquigarrow$ T1.WB



Is it Conflict Serializable?

Schedule: Interleave T1 & T2 & T3



≡ ?

- T1 → T2 → T3
- T2 → T1 → T3
- T3 → T1 → T2
- T2 → T3 → T1
- T1 → T3 → T2
- T3 → T2 → T1

All the possible serial schedules

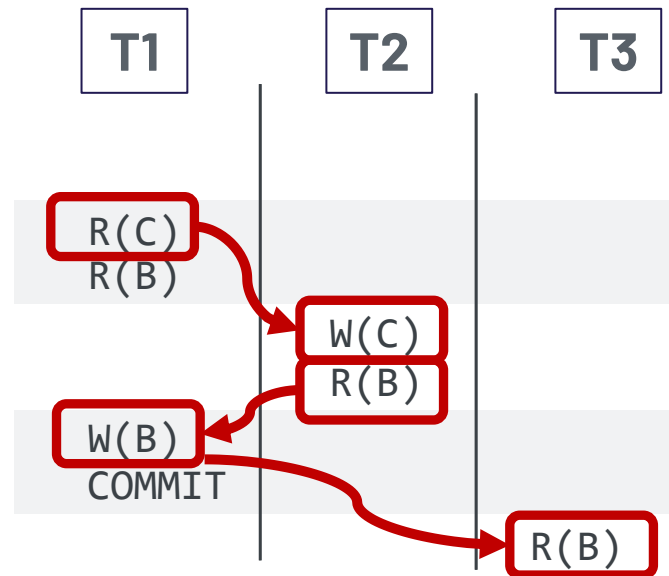


Is it Conflict Serializable? Use a Dependency Graph!

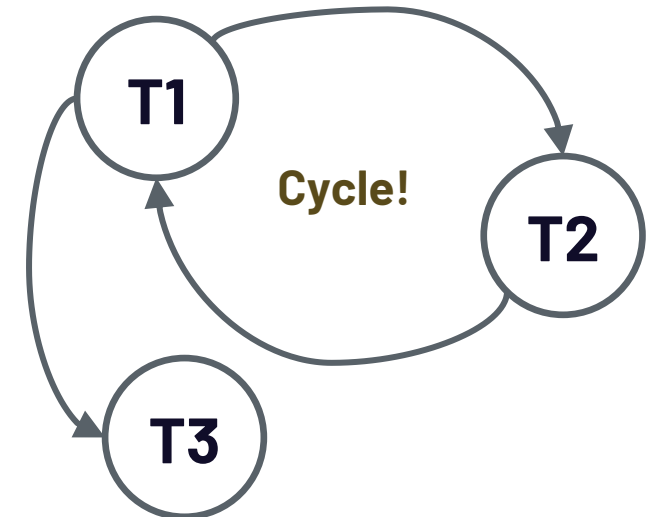
A dependency graph is a simple model to detect when a schedule is conflict serializable.

- It has a node for every transaction
- We put an **edge** between T_i and T_j if there is an operation O_i in T_i that has a conflict with operation O_j and O_i appears before O_j

A schedule is conflict serializable if and only if its dependency graph has no direct cycles



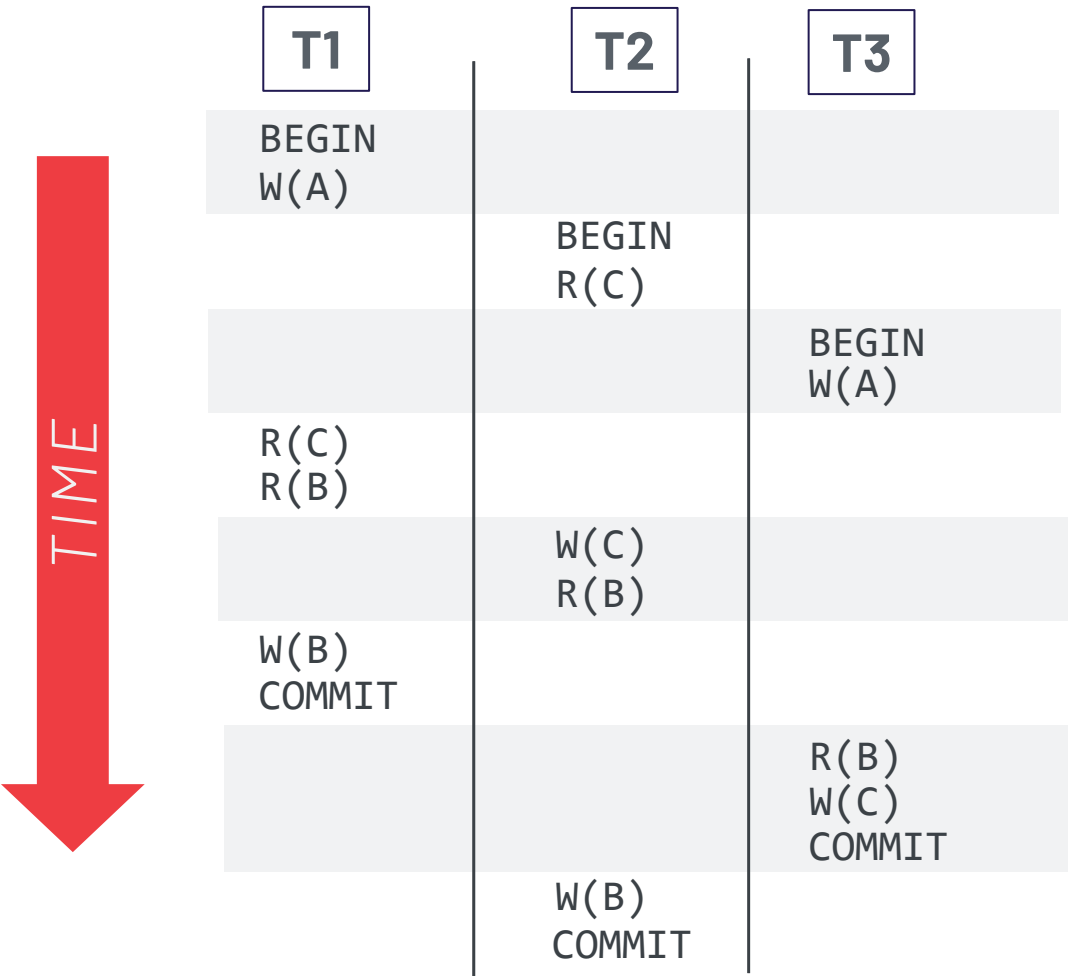
Also called
precedence graph!



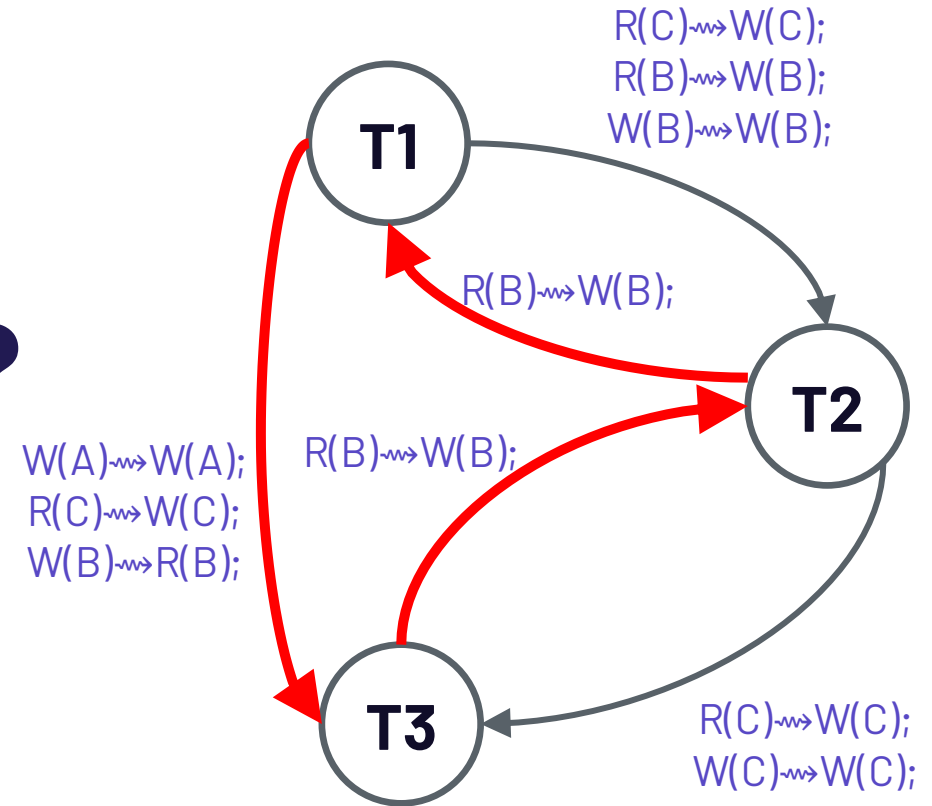


Is it Conflict Serializable?

Schedule: Interleave T1 & T2 & T3



≡ ?

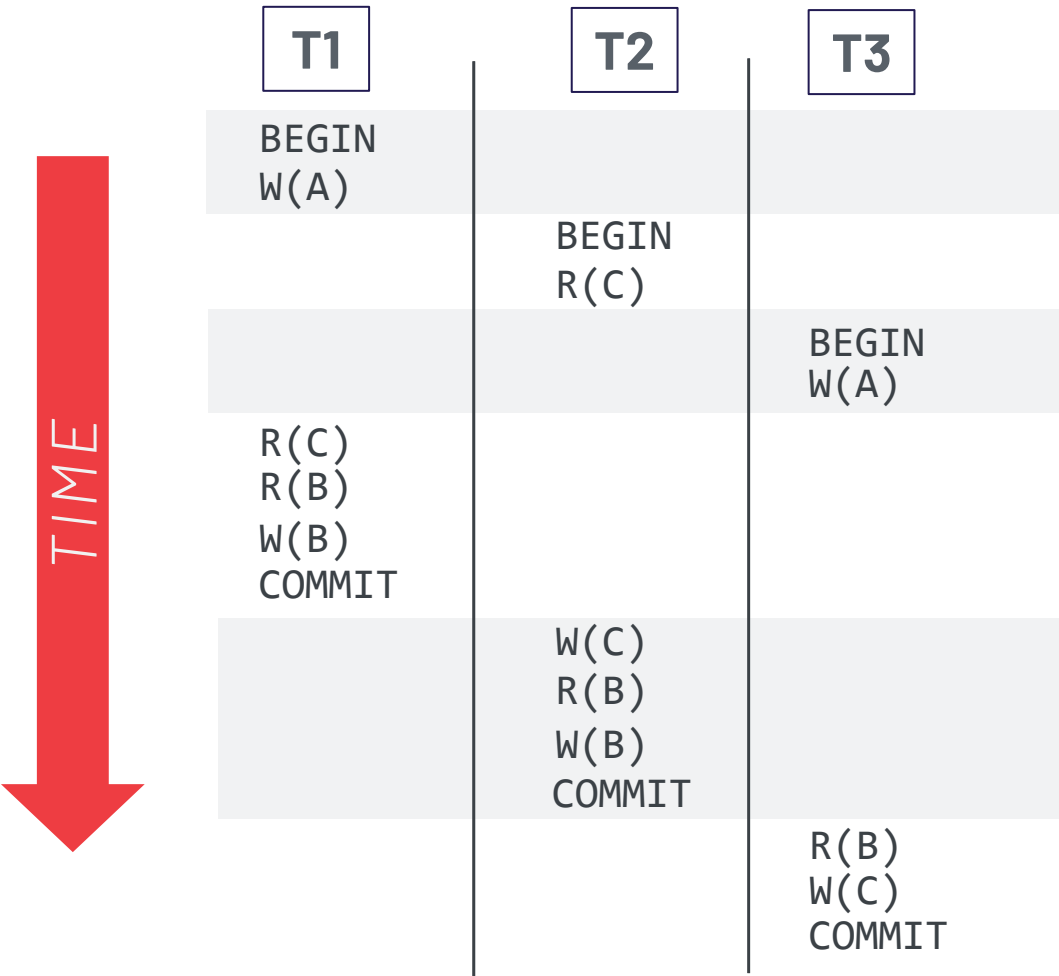


There is a directed cycle



Is it Conflict Serializable?

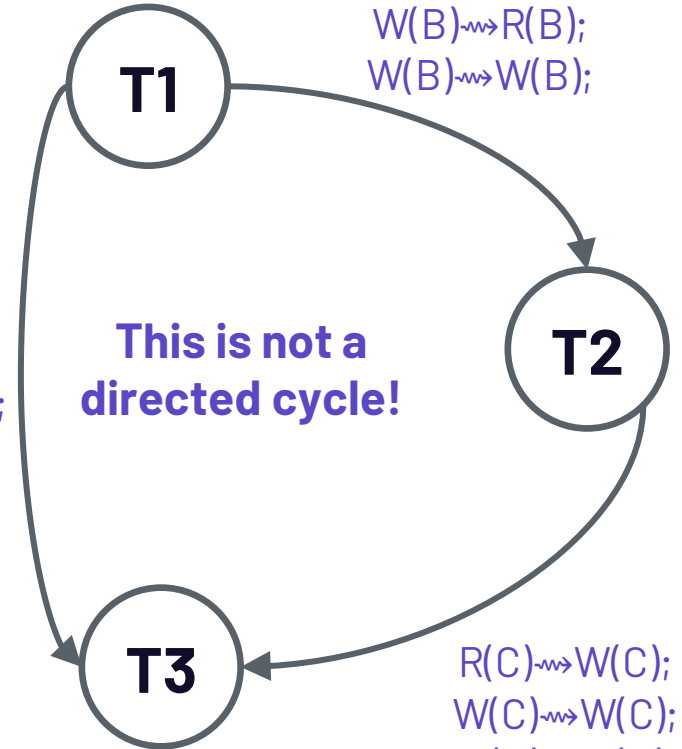
Schedule: Interleave T1 & T2 & T3



≡ ?

$W(A) \rightsquigarrow W(A);$
 $R(C) \rightsquigarrow W(C);$
 $W(B) \rightsquigarrow R(B);$

**This is not a
directed cycle!**



Equivalent to: T1 → T2 → T3 ✓

NOT Equivalent to: T1 → T3 → T2 ✗

Outline

1. Concurrency Control

- ▶ Transactions ★
- ▶ ACID Properties ★
- ▶ Serial Execution ★
- ▶ Interleaving Transactions

2. Scheduler

- ▶ Serial Schedule
- ▶ Equivalent Schedule
- ▶ Serializable Schedule ★
- ▶ Conflicting Transactions
- ▶ Conflict Serializability ★

3. Scheduling & Isolation Level

- ▶ Locks: Exclusive/Shared ★
- ▶ 2-Phase Locking basics ★
- ▶ 2-Phase Locking details
- ▶ Strong Strict 2-Phase locking
- ▶ Deadlocks ★
- ▶ Isolation Levels

4. Durability

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

Chapters 17, 18 without 18.10, 19 without 19.7-19.10

- from Database System Concepts 7th edition ; Silberschatz et al.

Intro to Database Systems (15-445/645), Andy Pavlo CMU

- <https://15445.courses.cs.cmu.edu/fall2022/>



Enforcing Serializable Executions

We need a PROTOCOL that ensures our schedules are conflict serializable ***while*** we are executing them!

(we also check, if at some point we make a mistake, we rollback)



We cannot wait to have all transactions before taking a decision!

What we have seen up to now can only tell us if a schedule “WAS” executed in a safe way...
...we need something that allow us to execute them correctly step by step.

Strict 2-Phase locking is a protocol that generates automatically and incrementally correct conflict serializable interleaving schedules

(but it has risks, see later!)

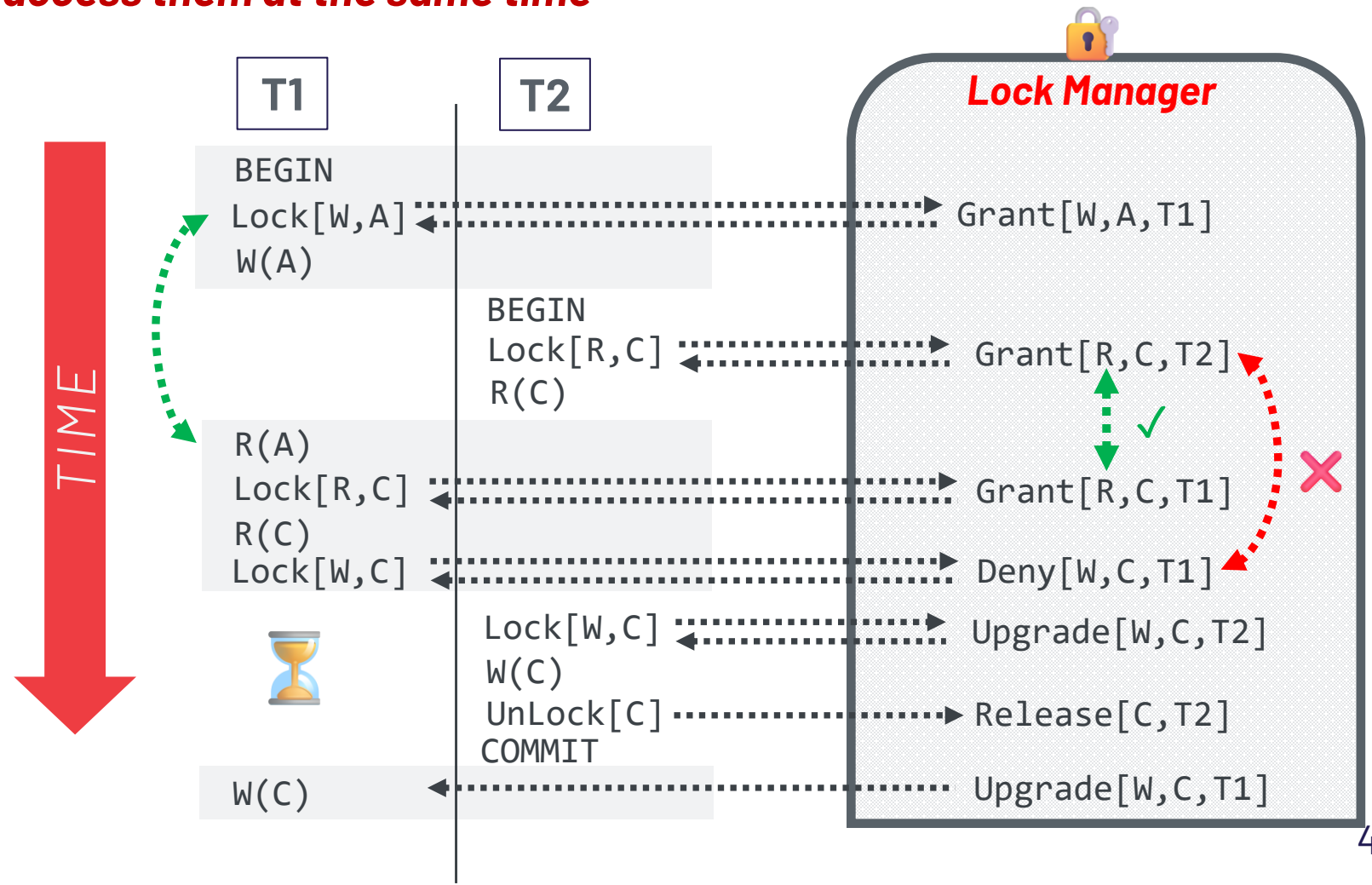


Lock and Lock Manager

We add a new component: **the Lock manager keeps tracks of which transactions is using which resource and decides who can access them at the same time**

**Operations of T1 & T2
arrive interleaved**

**Each transaction needs to
"ask permission" before
accessing some resource**





Locks vs. Latches

There are 2 concepts in DBMS that are used when dealing with concurrent access:

Locks and Latches, they are similar but used in different cases!

	Locks	Latches
Keep Separate	User Transactions	Threads
Protect	Database Contents (durable data)	In-Memory Data Structures (transient data)
During	Entire Transactions	Critical Sections in query operators
Types	Shares, Exclusive ... and others..	Read/Write
Handle Deadlock	Detection & Resolution	Avoidance
...using	Waits-for, Timeout, Abort	Coding Discipline
Stored in	Lock Manager	Protected in memory Data Structure



Basic Types of Locks

There are 2 basic types of locks

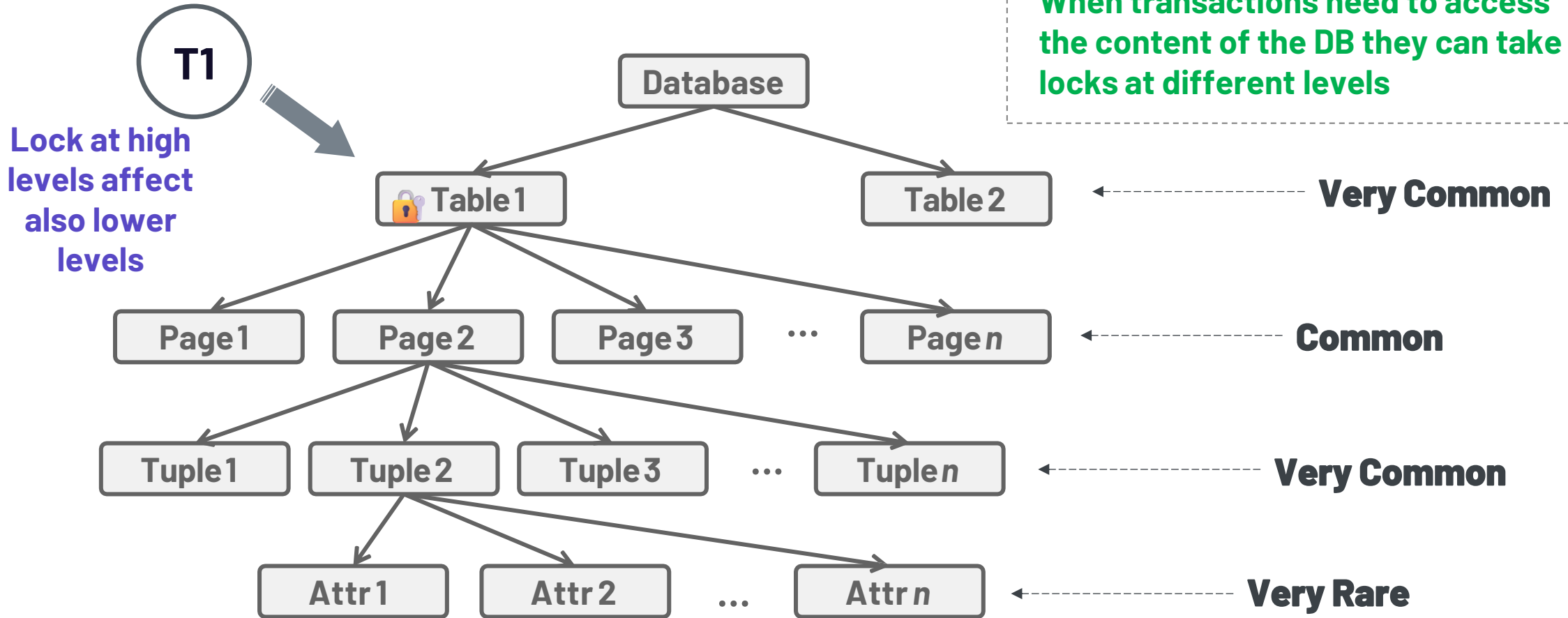
Shared [S-LOCK]: shared **locks are for reads**, to allow R-R situations and avoid R-W

Exclusive [X-LOCK]: exclusive **locks are for writes**, to avoid both W-R and W-W cases

Compatibility Matrix

		<i>can you get this lock on X?</i>	
		Shared	Exclusive
<i>if some other transactions has this lock on X</i>	Shared	✓	✗
	Exclusive	✗	✗

Database Lock Hierarchy



Not all systems support locks
at all levels

Compatibility of lock modes

The following table shows the compatibility of any two modes for page and row locks. No question of compatibility arises between page and row locks, because a partition or table space cannot use both page and row locks.

Table 1. Compatibility matrix of page lock and row lock modes

Lock mode	Share (S-lock)	Update (U-lock)
Share (S-lock)	Yes	Yes
Update (U-lock)	Yes	No
Exclusive (X-lock)	No	No

Compatibility for table space locks is similar to the compatibility of lock modes for partition, table space, or table locks.

Table 2. Compatibility of table and table space (or partition) lock modes

Lock Mode	IS	IX	S
IS	Yes	Yes	Yes
IX	Yes	Yes	No
S	Yes	No	Yes
U	Yes	No	Yes
SIX	Yes	No	No
X	No	No	No



Existing granted mode

Requested mode

Intent shared (IS)

Shared (S)

Update (U)

Intent exclusive (IX)

Shared with intent exclusive (SIX)



Table 13-3 Summary of Table Locks

SQL Statement	Mode of Table Lock	Lock Modes Permitted?					
		IS	IX	S	SIX	X	
SELECT...FROM table...	none	Y	Y	Y	Y	Y	Y
INSERT INTO table...	RX	Y	Y	N	N	N	N
UPDATE table...	RX	Y*	Y*	N	N	N	N
DELETE FROM table...	RX	Y*	Y*	N	N	N	N
SELECT...FROM table FOR UPDATE OF...	RS	Y*	Y*	Y*	Y*	N	N
LOCK TABLE table IN ROW SHARE MODE	RS	Y	Y	Y	Y	N	N
LOCK TABLE table IN ROW EXCLUSIVE MODE	RX					N	N
LOCK TABLE table IN SHARE MODE	S	Y	N	Y	N	N	N
LOCK TABLE table IN SHARE ROW EXCLUSIVE MODE	SRX	Y	N	N	N	N	N
LOCK TABLE table IN EXCLUSIVE MODE	X	N	N	N	N	N	N



Table 13.2. Conflicting Lock Modes

Requested Lock Mode	Existing Lock Mode					
	ACCESS	SHARE	ROW SHARE	ROW EXCL.	SHARE	UPDATE EXCL.
ACCESS SHARE						X
ROW SHARE					X	X
ROW EXCL.					X	X
SHARE UPDATE EXCL.				X	X	X
SHARE				X	X	X
SHARE ROW EXCL.			X	X	X	X
EXCL.			X	X	X	X
ACCESS EXCL.	X		X	X	X	X



Table-level lock type compatibility is summarized in the following matrix

	X	IX	S	IS
X	Conflict	Conflict	Conflict	Conflict
IX	Conflict	Compatible	Conflict	Compatible
S	Conflict	Compatible	Compatible	Compatible
IS	Conflict	Compatible	Compatible	Compatible





Concurrency Control Protocol with Locks

When using locks, the protocol is the following

1. **Transaction request** *locks on resource (or upgrades from S to X)*
2. **Lock manager** *inspect the request and **decides** to grant or deny the request*
3. **Lock Manager updates** *internal register: keeping track of transactions waiting*
4. **Transactions denied a request wait** *before proceeding*
5. **Transactions done with a resource inform the Lock Manager** *when releasing the lock*

Two-phase locking (2PL) is a concurrency control protocol that determines when transactions can decide to lock and unlock resources

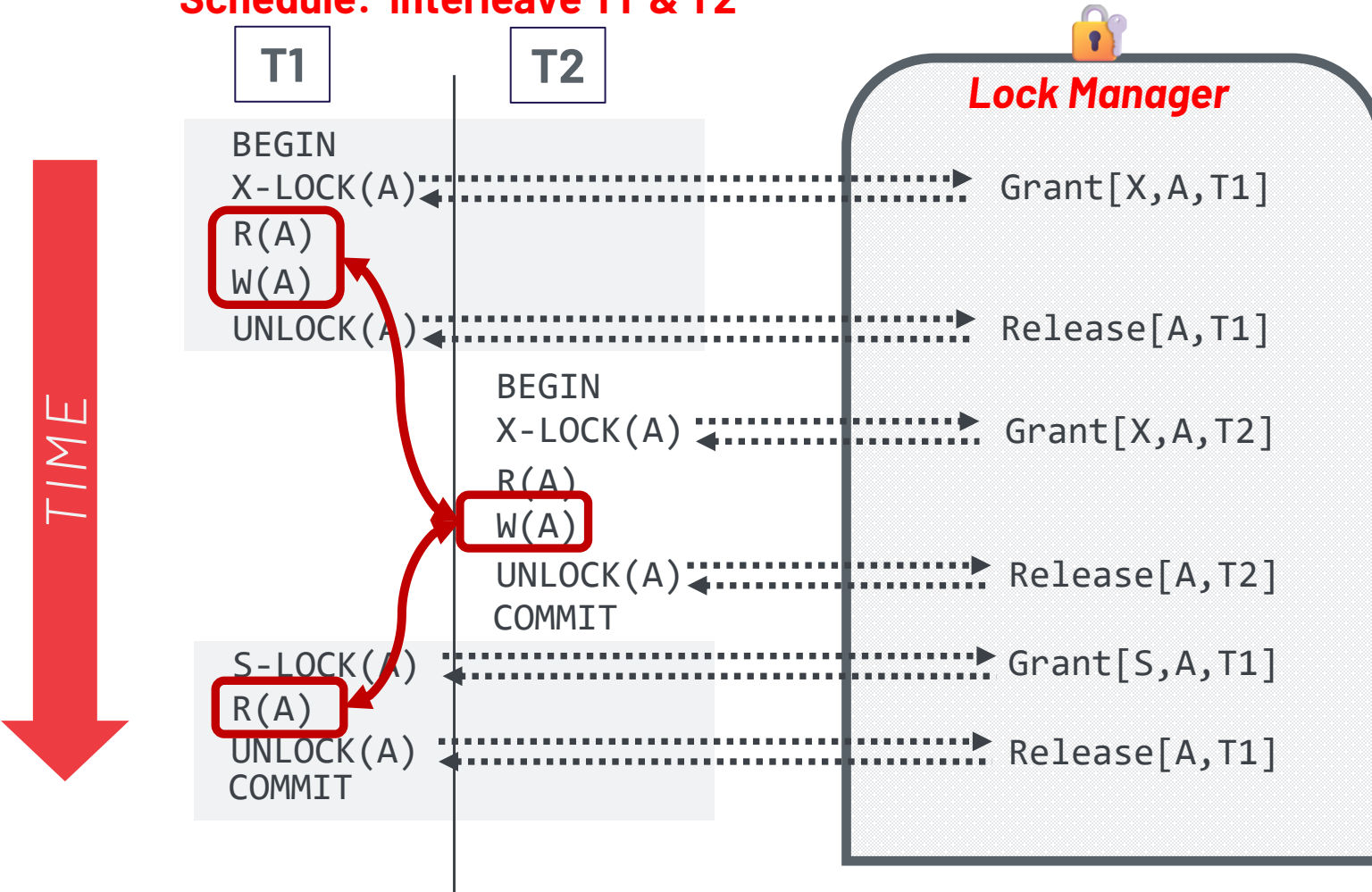
→ **When do T1 should request a lock? Which lock? When do T1 should signal unlock?**

2PL is a “pessimistic” CC Protocol: things will go badly, so we are conservative



Is it enough to use locks?

Schedule: Interleave T1 & T2



Using Locks is not enough,
we need a protocol that
ensures that we are using
them in the correct way!



The protocol is used
by the system,
SQL does not expect
you to "ask" for locks!

But you can provide hints!



Two-Phase Locking (2PL)

1. Phase #1: Growing:

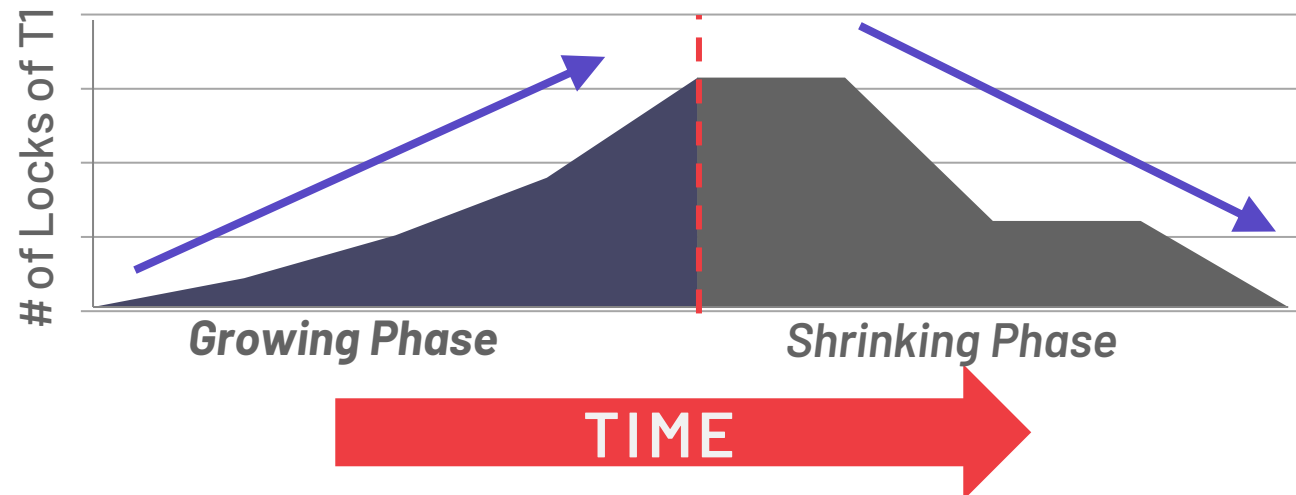
- Each transaction **request the locks** that it needs or to upgrade
- **The lock manager grants/denies** request and makes transaction wait
- **No lock is released** while in this phase

2. Phase #2: Shrinking

- **The transactions release** or downgrades locks obtained in Phase #1
- **No new lock can be acquired** in this phase

AS SOON AS the transaction RELEASE a lock, they cannot get any new one, we start Phase 2

Visualizing a **correct** 2PL execution



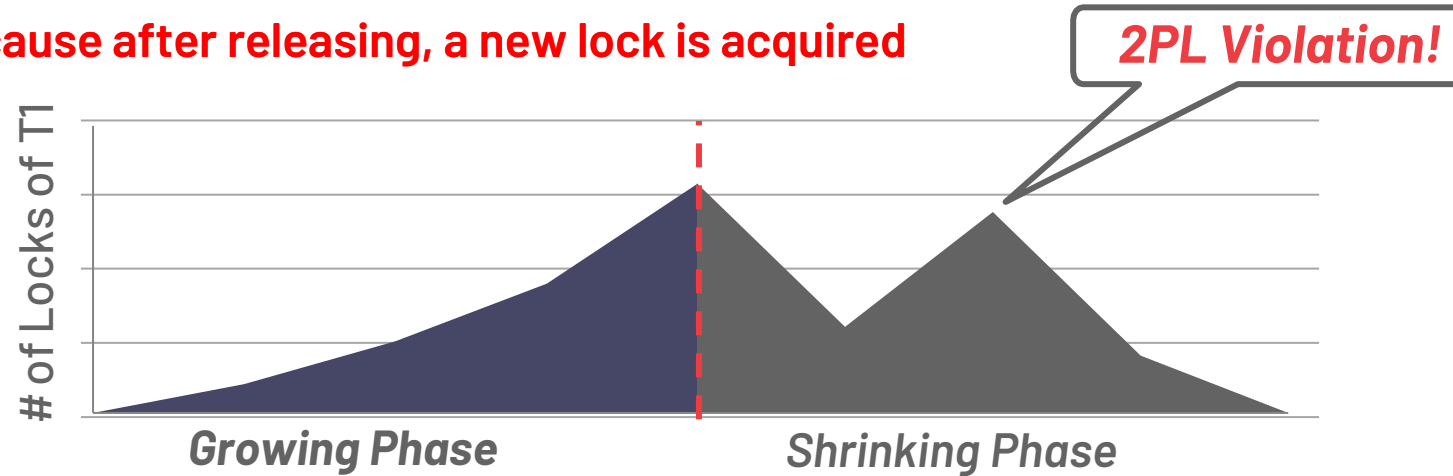


Two-Phase Locking (2PL) – never lock after unlock

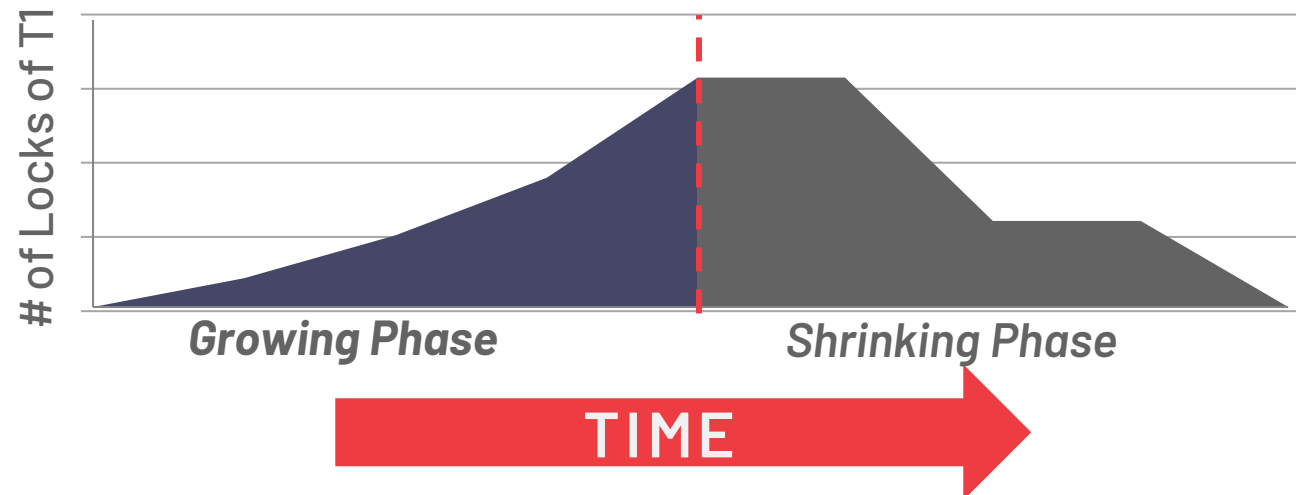
A transaction cannot acquire or upgrade locks after it started unlocking

The first is wrong because after releasing, a new lock is acquired

Visualizing a
wrong 2PL
execution



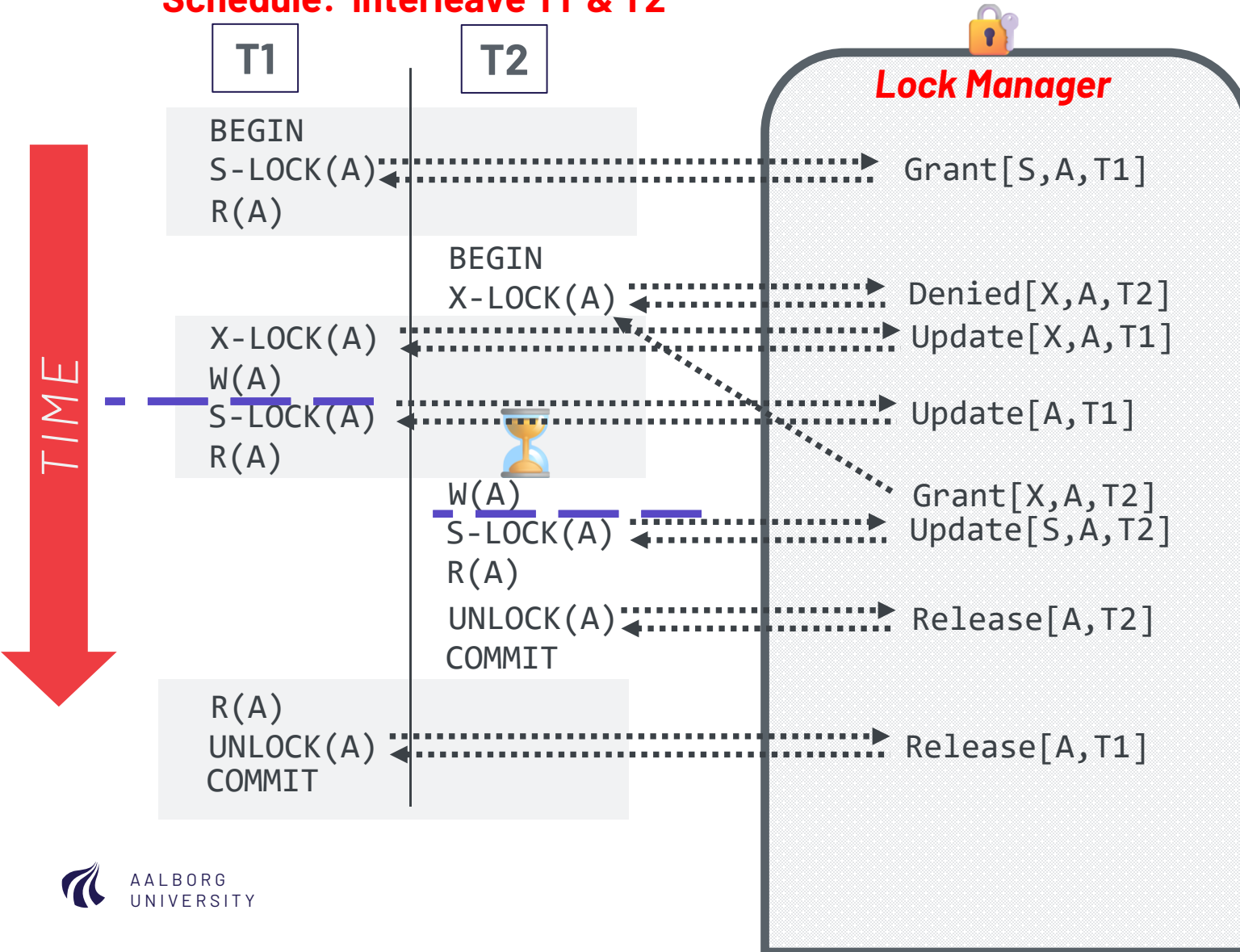
Visualizing a
correct 2PL
execution





2PL Example and Wait before Commit

Schedule: Interleave T1 & T2



To make it work we ask locks as early as possible, and we do not release any lock until we have all we need



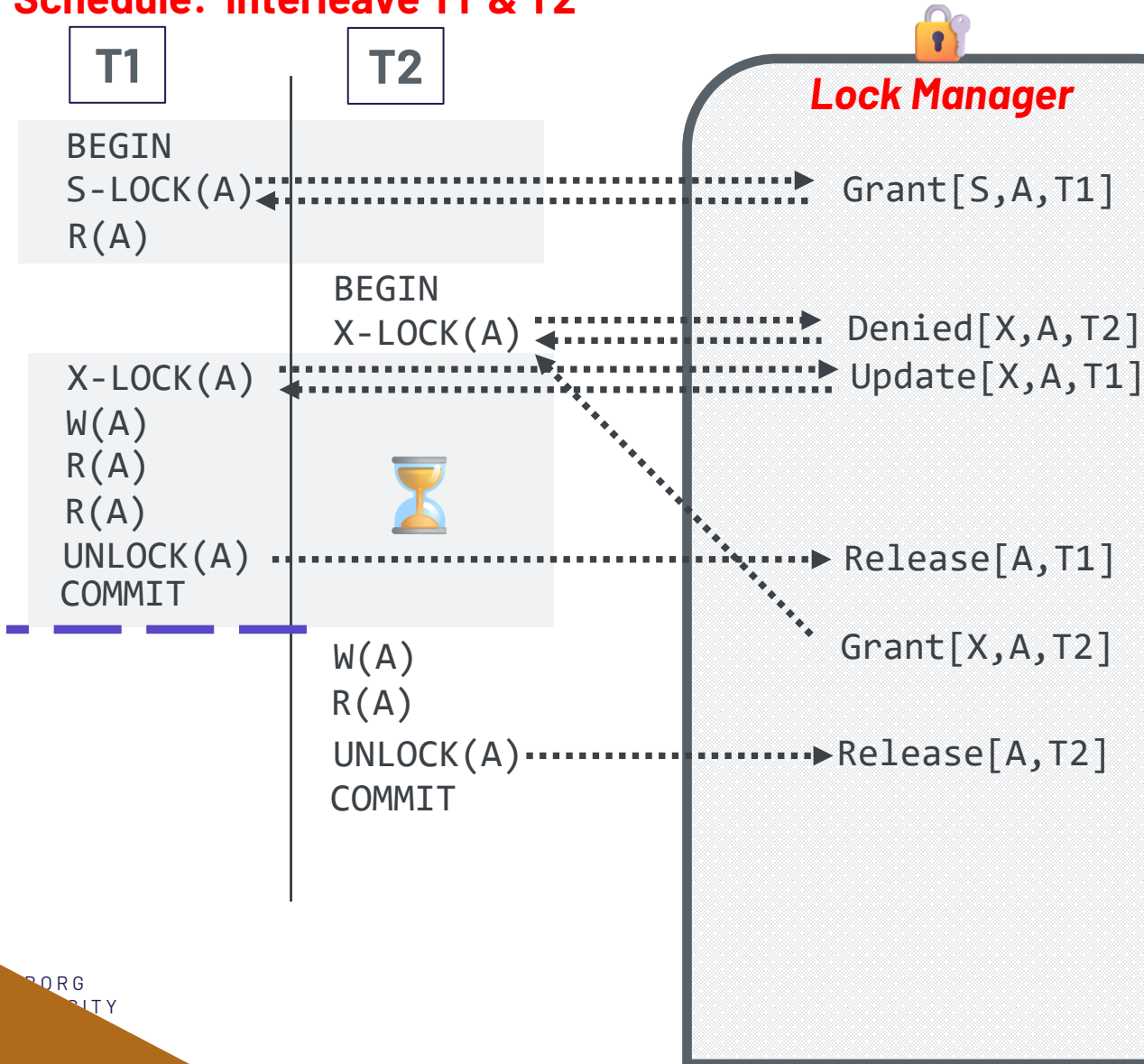
T2 has to wait before returning OK to the user, to see if T1 can commit

WARNING: here we have applied 2PL, but we decided to DOWNGRADE X-LOCK(A)
THIS CAN CREATE SOME ANOMALIES
We need a STRICTER PROTOCOL
(see next slide)



2PL Example and Wait before Commit – NO DOWNGRADE

Schedule: Interleave T1 & T2



To make it work we ask locks as early as possible, and we do not release any lock until we have all we need
ALSO: NO DOWNGRADE



T2 has to wait before returning OK to the user, to see if T1 can commit



Strong Strict 2PL

1. Phase #1: Growing:

- **No lock is released** while in this phase
- **NO LOCK IS DOWNGRADED**

A schedule is “strict” if a value written by a txn T1 is not read or overwritten by other txns until T1 finishes

2. Phase #2: Shrinking

- **No new lock can be acquired** in this phase
- **No operation other than Unlock happens in this phase**
- **NO DOWNGRADE**

To avoid completely all issues “Strong Strict 2PL” enforces that a transaction is allowed to release locks only right before COMMIT



Simple Rules to apply 2PL

- 1. We acquire the lock as soon as we need it**
- 2. We can UPGRADE locks**
- 3. We do not DOWNGRADE locks**
- 4. We release locks when we are done BUT ONLY WHEN WE DO NOT
NEED TO ACQUIRE NEW LOCKS $\Rightarrow$ REQUIRES TO PREDICT THE FUTURE**

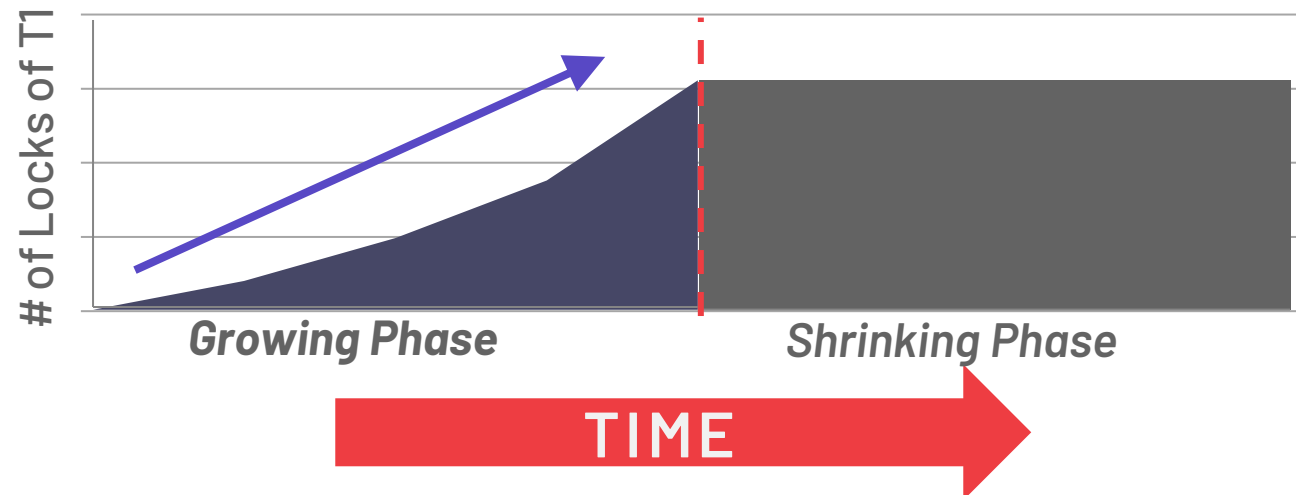


Observations on 2PL and Strong Strict 2PL

1. **it's conservative:** there are sometimes serializable schedule that 2PL will ignore/prohibit
2. **This will limit performance/parallelism, but is the only one ensuring correctness**
3. **Strong Strict 2PL is required to avoid dirty reads**
4. **May lead to deadlocks!**

Strong Strict 2PL never releases until the very end, no gradual shrinking

Visualizing a
correct
Strong Strict
2PL execution





Simple Rules to apply Strong Strict 2PL

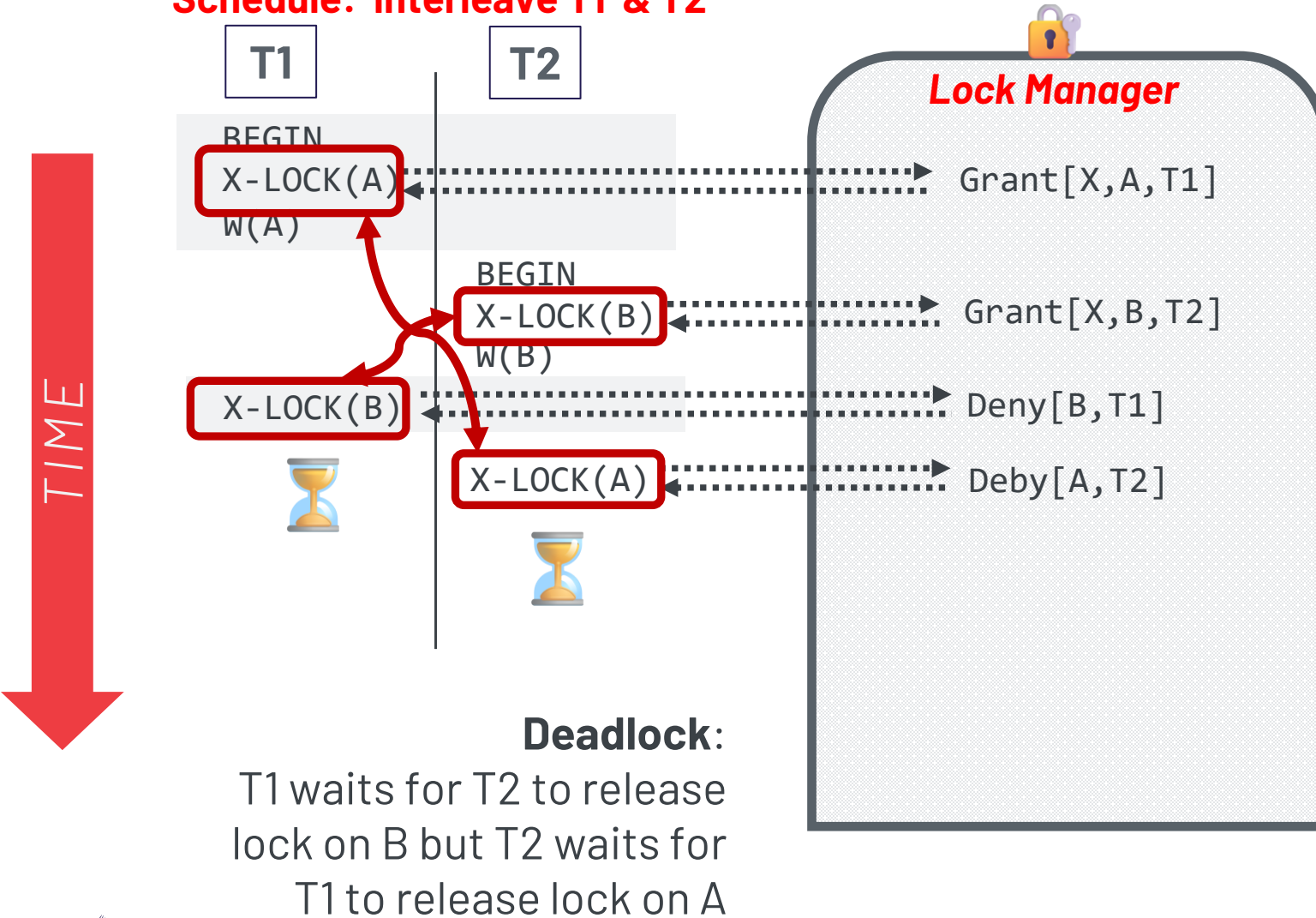
- 1. We acquire the lock as soon as we need it**
- 2. We can UPGRADE locks**
- 3. We do not DOWNGRADE locks**
- 4. We release locks ONLY when we COMMIT**
⇒ DOES NOT REQUIRES TO PREDICT THE FUTURE

“Strong Strict 2PL” is the easiest to apply, but can incur in deadlocks



A deadlock

Schedule: Interleave T1 & T2



Using 2PL and especially Strong Strict 2PL we can incur into deadlocks!



To deal with deadlock we can either:

- (1) Detect Deadlock and kill one of the transactions causing it
- (2) Prevent deadlocks and abort transactions asking too much

Timestamp Ordering Concurrency Control

Goal: avoid use of locks because expensive

Determine serializability order of a txn before they execute (optimistic)

- transactions can read/write without locks but at the end we **check timestamps to not** overwrite operations “from the future”: **it checks only at the end** if there has been a conflict

Optimistic Concurrency Control protocol:

- first use a **private workspace** (copy data for each transaction), then **when done apply changes to global DB** if your timestamp is compatible

Until now we considered writes means update: locks and timestamp work only for things that exist when the transaction starts running

- **insert new things and delete things**, there are **phantom reads**: use index locks to avoid inserts/deletes

Isolation Levels

Serializability is safe but costly

Many systems **use a weaker version** of concurrency control, which means they allow some errors to happen

**HIGH
ISOLATION**



**LOW
ISOLATION**

- **SERIALIZABLE:** no phantoms, all reads repeatable, no dirty reads
- **REPEATEDB READS:** phantoms may happen
- **READ COMMITTED:** phantoms may happen and unrepeatable reads
- **READ UNCOMMITTED:** allows dirty read in addition to above

Additional Option:

Multi Version Concurrency Control (MVCC) similar level of guarantees as given by "repeatable reads"

https://www.youtube.com/watch?v=ht8Tx6Dwn_E



ISOLATION LEVELS

What can happen with each different isolation level?

ISOLATION LEVEL	ISSUES THA CAN HAPPEN			
		<i>Dirty Read</i>	<i>Unrepeatable Read</i>	<i>Phantom</i>
	SERIALIZABLE	No	No	No
	REPEATABLE READ	No	No	Maybe
	READ COMMITTED	No	Maybe	Maybe
	READ UNCOMMITTED	Maybe	Maybe	Maybe

Note that some system do not have SERIALIZABLE by default, you need to activate it

Outline

1. Concurrency Control

- ▶ Transactions ★
- ▶ ACID Properties ★
- ▶ Serial Execution ★
- ▶ Interleaving Transactions

2. Scheduler

- ▶ Serial Schedule
- ▶ Equivalent Schedule
- ▶ Serializable Schedule ★
- ▶ Conflicting Transactions
- ▶ Conflict Serializability ★

3. Scheduling & Isolation Level

- ▶ Locks: Exclusive/Shared ★
- ▶ 2-Phase Locking basics ★
- ▶ 2-Phase Locking details
- ▶ Strong Strict 2-Phase locking ★
- ▶ Deadlocks ★
- ▶ Isolation Levels

4. Durability

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

Chapters 17, 18 without 18.10, 19 without 19.7-19.10

- from Database System Concepts 7th edition ; Silberschatz et al.

Intro to Database Systems (15-445/645), Andy Pavlo CMU

- <https://15445.courses.cs.cmu.edu/fall2022/>



Properties for a Transaction: ACID

Atomicity

All actions in txn happen, or none happen.

"All or nothing..."

if a debit is made successfully from one account, the corresponding credit is made to the other account.

Consistency

If each txn is consistent and the DB starts consistent, then it ends up consistent.

"It looks correct to me..."

the total value of funds across accounts is the same at the start and end of each transaction.

Isolation

Execution of one txn is isolated from that of other txns.

"All by myself..."

another transaction sees the transferred funds in one account or the other, but not in both, nor in neither.

Durability

If a txn commits, its effects persist.

"I will survive..."

the changes made to each account will not be reversed.



Crash Recovery

To ensure durability:

1. Recovery algorithms are techniques to **ensure the database consistency**, atomicity, and **durability despite failures**
2. Recovery algorithms have two parts
 1. **Actions during** normal txn processing to ensure the DBMS can recover from failure
 2. **Actions after** failure to recover the database state

Overall solution: Use a log + checkpoints:

1. **write all your operations in a log of operations before anything else**
2. **if you fail, compare the operation in the log with what has been completed in the DB**

Write-Ahead Log

Maintain a log file separate from data files that contains the changes that txns make to database.

- Assume that the log is on stable storage (storage that is safe and secure).
- Log contains enough information to perform the necessary undo and redo actions to restore the database.

DBMS must write to disk the log file records that correspond to changes made to a database object before it can flush that object to disk.

All log records pertaining to an updated page are written to non-volatile storage before the page itself is over-written in non-volatile storage.

A txn is not considered committed until all its log records have been written to stable storage.

Outline

1. Concurrency Control

- ▶ Transactions ★
- ▶ ACID Properties ★
- ▶ Serial Execution ★
- ▶ Interleaving Transactions

2. Scheduler

- ▶ Serial Schedule
- ▶ Equivalent Schedule
- ▶ Serializable Schedule ★
- ▶ Conflicting Transactions
- ▶ Conflict Serializability ★

3. Scheduling & Isolation Level

- ▶ Locks: Exclusive/Shared ★
- ▶ 2-Phase Locking basics ★
- ▶ 2-Phase Locking details
- ▶ Strong Strict 2-Phase locking
- ▶ Deadlocks ★
- ▶ Isolation Levels

4. Durability

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

Chapters 17, 18 without 18.10, 19 without 19.7-19.10

- from Database System Concepts 7th edition ; Silberschatz et al.

Intro to Database Systems (15-445/645), Andy Pavlo CMU

- <https://15445.courses.cs.cmu.edu/fall2022/>



The whole picture

- **How do we design a DB and organize it in a DBMS so that the queries are executed efficiently?**
- **What makes a query slow or fast?** Expensive to compute or efficient?
- What ensures that our **data is stored safely and correctly** also when **multiple queries** are running at the same time?

